

Option 3 of the Course Project: Cryptographic Algorithms & Modular Arithmetic



Project Option 3: Cryptographic Algorithms and Modular Arithmetic

Implementing and Analyzing Number-Theoretic Cryptography

Explore the mathematical underpinnings and practical implementation of cryptographic algorithms based on modular arithmetic.



Requirements

Choose one of the following **cryptographic systems or protocols**:

1. RSA Encryption and Decryption
2. Diffie–Hellman Key Exchange
3. Shamir's Secret Sharing (with at least 3 participants and 2 needed to reconstruct the secret)

Each option **must include**:



Written Report (PDF or DOCX)

- **Clear overview** of the selected cryptographic system and its purpose
- **Mathematical background**:
 - **Modular arithmetic, Euler's theorem, modular inverses, prime factorization**, etc.
- **Detailed pseudocode** for key parts of the algorithm (e.g., key generation, encryption/decryption, or reconstruction)
- **Worked-out numerical example** using realistic small primes (for RSA and Diffie-Hellman)
- **Security analysis**: potential vulnerabilities or assumptions (e.g., difficulty of factoring, discrete log problem)
- **Real-world relevance** and example use cases



Code Submission

- **Implementation** in a programming language (e.g., Python, JavaScript, C, Java)
- The code must include:
 - **Input and output** for all algorithm phases (e.g., key generation, encryption/decryption)
 - **Proper comments** explaining steps
- **Test your code** with at least one full example run (**with link to online code implementation platform**)

Recommended platforms: Replit, Google Colab, GitHub, or Jupyter Notebook



Video Presentation (5–8 minutes)

- Present the algorithm, key math concepts, and step-by-step example
- Explain how your code works (screencast or slide-based)
- Include visuals like modular arithmetic tables or diagrams (e.g., flow of Diffie-Hellman exchange)
- Reflect on:
 - What challenges you faced
 - Why this algorithm is important in modern security



Additional Challenge Requirement

- For **RSA**: Implement **key generation**, **encryption**, and **decryption**, and explain **why Euler’s totient function $\phi(n)$** is used.
- For **Diffie–Hellman**: Show the **full exchange between two parties**, with intermediate steps and shared secret verification.
- For **Shamir’s**: Use at least **3 shares**, reconstruct the secret from any **2 shares**, and **explain Lagrange interpolation** over finite fields (mod p).



Grading Rubric (Total: 100 points)

Category	Points	Criteria
Mathematical Understanding	25	Demonstrates clear grasp of core concepts; explains

Category	Points	Criteria
		problem and theory clearly and accurately
Algorithm/Pseudocode	15	Includes correct and logical pseudocode for the algorithm or method
Code Implementation	15	Working implementation provided; well-commented and readable
Written Submission Quality	15	Clearly written, well-organized, and complete (all parts included)
Video Presentation	20	Clear delivery; includes visual demonstration and conceptual explanation; within time limit
Creativity/Engagement	10	Originality, effort, and engagement in the topic (visuals, examples, reflection)